Module : Composant Android (M205)
TP N° 9

Année de Formation 2024/2025
Filière : Développement Mobile Groupe : DEVOAM 201 - DEVOAM 202 Niveau : 2ème année

Exercice 1 :

L'application permettra aux utilisateurs de suivre leur activité physique, d'enregistrer des entraînements, de recevoir des rappels pour des objectifs d'activité, et d'utiliser des services comme la localisation pour une expérience enrichie.

- **Gérer des tâches asynchrones** pour synchroniser les données avec un serveur en arrière-plan sans interférer avec l'expérience utilisateur.
- Utiliser **WorkManager** et **JobScheduler** pour effectuer des tâches récurrentes et gérer les conditions comme la connectivité réseau ou l'état de la batterie.
- **Manipuler des Threads** et des **Coroutines Kotlin** pour exécuter des tâches longues sans bloquer le thread principal de l'application.
- **Gérer les permissions** nécessaires, notamment la **localisation**, le **réseau**, et le **Bluetooth**, en utilisant les meilleures pratiques pour assurer la confidentialité et la performance.

Énoncé des fonctionnalités du projet :

1. **Suivi des Entraînements :**
    - Permettre à l'utilisateur de démarrer, de suivre et d'arrêter une session d'entraînement (course à pied, marche, vélo, etc.).
    - Enregistrer les données d'entraînement (distance, calories brûlées, durée) et les afficher dans une interface utilisateur.
    - Synchronisation périodique des données d'entraînement avec un serveur (ou stockage local).
2. **Rappels Géolocalisés :**

- Envoyer des rappels à l'utilisateur pour l'encourager à faire de l'exercice lorsque celui-ci se trouve dans une zone géographique déterminée (par exemple, un parc ou une salle de sport).
- Utiliser **WorkManager** pour planifier des rappels géolocalisés en fonction de la position GPS de l'utilisateur.

3. **Tâches en Arrière-plan avec WorkManager et JobScheduler :**
   - Utiliser **WorkManager** pour effectuer des tâches de synchronisation des données (par exemple, télécharger les statistiques d'entraînement) même lorsque l'application est en arrière-plan.
   - Implémenter **JobScheduler** pour planifier des tâches qui s'exécutent lorsque le réseau Wi-Fi est disponible ou que l'appareil est suffisamment chargé.
4. **Gestion des Permissions :**
   - Demander les permissions nécessaires pour accéder à la **localisation** (pour le suivi GPS) et au **réseau** (pour la synchronisation des données).
   - Implémenter un workflow de demande de permissions (avec message explicatif) pour que l'utilisateur accorde les permissions de manière claire et transparente.
   - Utiliser **EasyPermission** pour simplifier la gestion des permissions.

Exercice 2 :

1. Découverte des tâches asynchrones

1. Quelles sont les différentes manières de gérer des tâches asynchrones dans une application Android ?
2. Comment utiliser les **Coroutines Kotlin** pour effectuer des tâches en arrière-plan sans bloquer l'interface utilisateur ?
3. Qu'est-ce que le problème du "Main Thread" en Android, et comment les tâches asynchrones peuvent-elles le résoudre ?

4. Quelle est la différence entre les **Threads** et les **Coroutines** en Kotlin ? Quand faut-il utiliser l'un ou l'autre ?

2. Utilisation du WorkManager

5. Qu'est-ce que le **WorkManager** en Android et dans quels cas est-il recommandé de l'utiliser ?
6. Comment créer et configurer une tâche en arrière-plan avec **WorkManager** pour une exécution immédiate (OneTimeWorkRequest) ?
7. Comment configurer une tâche qui s'exécute périodiquement avec **WorkManager** (PeriodicWorkRequest) ?
8. Quelles sont les contraintes que l'on peut définir dans une tâche avec **WorkManager** (ex. : réseau, charge de la batterie) ?

3. Implémentation de JobScheduler

9. Quelle est la différence entre **WorkManager** et **JobScheduler** ? Quand faut-il privilégier l'utilisation de l'un par rapport à l'autre ?
10. Comment utiliser **JobScheduler** pour planifier une tâche qui ne s'exécute que lorsqu'un réseau Wi-Fi est disponible ?
11. Comment gérer les priorités et les conditions (comme la batterie, la connectivité réseau) avec **JobScheduler** ?

4. Manipulation des Threads avec Kotlin

12. Comment créer et gérer des **Threads** en Kotlin pour exécuter des tâches en arrière-plan dans une application Android ?
13. Quelle est l'importance des **Coroutines Kotlin** pour remplacer les **Threads** dans Android, et quelles sont les meilleures pratiques pour les utiliser ?
14. Comment synchroniser les données entre les différents threads en Kotlin sans causer de problèmes de concurrence ?

15. Que sont les **Dispatchers** dans Kotlin et comment les utiliser pour contrôler où une coroutine est exécutée (ex. : sur le thread principal ou un thread en arrière-plan) ?

5. Présentation du fichier Manifest.xml

16. Quel est le rôle du fichier **AndroidManifest.xml** dans une application Android ?
17. Comment déclarer des permissions dans le fichier **AndroidManifest.xml** pour accéder à la localisation, au Bluetooth et au réseau ?
18. Que sont les **permissions d'installation** (install-time permissions) et **permissions d'exécution** (runtime permissions) dans Android ? Comment les déclarer et les gérer dans le fichier manifeste ?

6. Utilisation des Install-time permissions

19. Quelles sont les permissions que l'on déclare dans le fichier **AndroidManifest.xml** et qui sont demandées au moment de l'installation de l'application ?
20. Comment déclarer correctement la permission d'accès à la **localisation** et au **réseau** dans le **AndroidManifest.xml** ?

7. Utilisation des Runtime permissions

21. Quelles sont les **runtime permissions** en Android et pourquoi doivent-elles être demandées après l'installation de l'application ?
22. Comment demander des permissions d'exécution telles que l'accès à la localisation, au réseau, au Bluetooth, et aux contacts ?
23. Quelle est la différence entre une permission de **type normale** et une permission de **type dangereuse** dans Android ?
24. Quelles sont les meilleures pratiques pour gérer le refus de permissions par un utilisateur ?

8. Utilisation des Workflow pour la demande de permissions

25. Comment implémenter un **workflow de demande de permissions** pour guider l'utilisateur dans le processus de demande (ex. : demander la permission de localisation avec un message explicatif) ?
26. Qu'est-ce qu'un **rationale** (raison) pour une permission dans Android, et comment afficher une explication avant de demander la permission ?

9. Utilisation du Permission de localisation

27. Comment demander l'accès à la **localisation** de manière à respecter les bonnes pratiques de confidentialité ?
28. Comment gérer les permissions d'accès à la **localisation fine** (GPS) et à la **localisation approximative** (réseau) dans une application Android ?
29. Quelles sont les limitations et les bonnes pratiques concernant l'utilisation de la localisation en arrière-plan ?

10. Présentation de la permission d'accès au réseau

30. Comment déclarer et demander la permission d'accès au **réseau** pour effectuer des opérations réseau dans votre application ?
31. Quelles sont les meilleures pratiques pour vérifier la connectivité réseau et gérer les erreurs de réseau dans une application Android ?

11. Présentation de la permission d'accès aux contacts de téléphone

32. Quelles sont les étapes pour demander et utiliser la permission d'accès aux **contacts téléphoniques** dans une application Android ?
33. Comment utiliser l'API des **Contacts** en Android pour afficher ou manipuler les contacts, tout en respectant les règles de confidentialité ?

12. Présentation de la permission d'accès au Bluetooth

34. Quelles permissions sont nécessaires pour accéder au **Bluetooth** dans Android et comment les déclarer dans le manifeste ?

35. Comment vérifier l'état du Bluetooth et se connecter à un appareil Bluetooth externe, comme un capteur de fréquence cardiaque, tout en gérant les permissions ?

13. Aperçu sur les bibliothèques de permissions existantes

36. Quelles sont les principales bibliothèques tierces disponibles pour faciliter la gestion des permissions dans une application Android ?
37. Quels sont les avantages d'utiliser une bibliothèque comme **EasyPermission** pour gérer les demandes de permissions ?

14. Focus sur la bibliothèque EasyPermission : Demandes d'accès

38. Comment intégrer la bibliothèque **EasyPermission** dans votre projet Android pour simplifier la gestion des permissions ?
39. Comment utiliser **EasyPermission** pour demander des permissions d'accès à la **localisation**, au **Bluetooth**, et aux **contacts** ?
40. Quels sont les avantages de **EasyPermission** par rapport à la gestion manuelle